43 WEB CODER

POWER OF TEAMWORK

Bài 1

Tổng quan về Spring MVC

MODULE 4

Mục tiêu



- Trình bày được ý nghĩa của các Framework
- Trình bày được tổng quan kiến trúc của Spring MVC
- Trình bày được cơ chế Dependency Injection
- Trình bày được ý nghĩa của Model trong Spring MVC
- Trình bày được DispatcherServlet
- Tạo được ứng dụng Spring MVC cơ bản
- Triển khai được ứng dụng Spring MCV cơ bản

Thảo luận

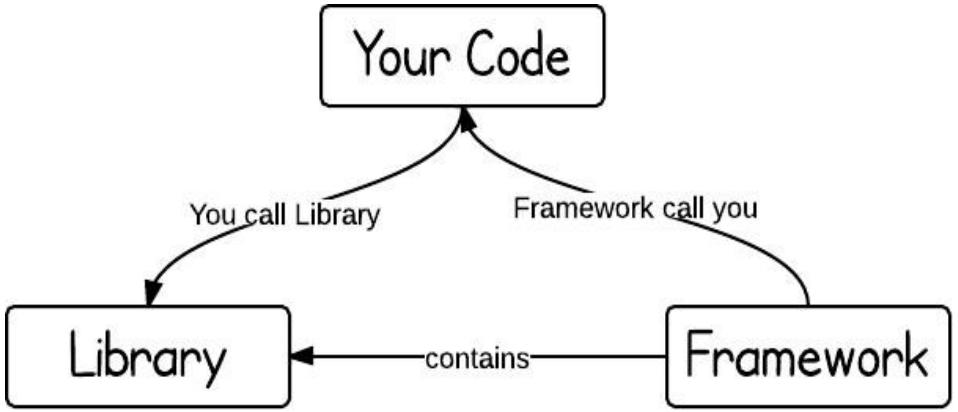
Framework

Framework

- Framework là các ứng dụng phần mềm có tính trừu tượng (abstraction), cung cấp các tính năng chung và thông dụng, có thể tuỳ biến để tạo nên những ứng dụng cụ thể khác nhau
- Mỗi framework cung cấp một phương pháp riêng biệt để xây dựng và triển khai ứng dụng
- Mỗi framework bao gồm một môi trường tổng thể, tái sử dụng được nhằm cung cấp các chức năng và công cụ để hỗ trợ quá trình phát triển ứng dụng

Framework vs Library

- Diểm khác biệt lớn nhất giữa Frameworkvà Libraryđóchínhlàcơ chế "Inversion of Control"
- Với Library: Ứng dụng nắm quyền điều khiển (control)
- Với Framework: Framework nắm quyền điều khiển



Lợi ích của Framework



- Framework giải quyết các vấn đề thông dụng, giúp lập trình viên tập trung vào xử lý nghiệp vụ
- Giúp tăng tốc độ phát triển
- Cung cấp môi trường làm việc tiêu chuẩn, giúp dễ giao tiếp giữa các bên khi cùng tham gia phát triển
- Các framework thường có cộng đồng phát triển lớn, các giải pháp đã được đánh giá và thử nghiệm, hệ sinh thái đầy đủ giúp nhanh chóng xây dựng được các giải pháp tuỳ biến



Thảo luận

Spring Framework

Spring Framework



- SpringFramework cungcấp một môhình đầy đủ cho việc phát triển và cấu hình các hệ thống Java lớn
- Các tính năng lõi:
  - **Core**: IoC container, Events, Resources, i18n, Validation, Data Binding, Type Conversion, SpEL, AOP.
  - **Testing**: Mock objects, TestContext framework, Spring MVC Test, WebTestClient.
  - **Data Access**: Transactions, DAO support, JDBC, ORM, Marshalling XML.
  - **Web Servlet**: Spring MVC, WebSocket, SockJS, STOMP messaging.
  - **Web Reactive**: Spring WebFlux, WebClient, WebSocket.
  - **Integration**: Remoting, JMS, JCA, JMX, Email, Tasks, Scheduling, Cache.
  - **Languages**: Kotlin, Groovy, Dynamic languages.

Các dự án Spring



- Spring IO Platform
- Spring Boot
- Spring Framework
- Spring Cloud Data Flow
- Spring Cloud
- Spring Data
- Spring Integration
- Spring Batch
- Spring Security
- ...



Thảo luận

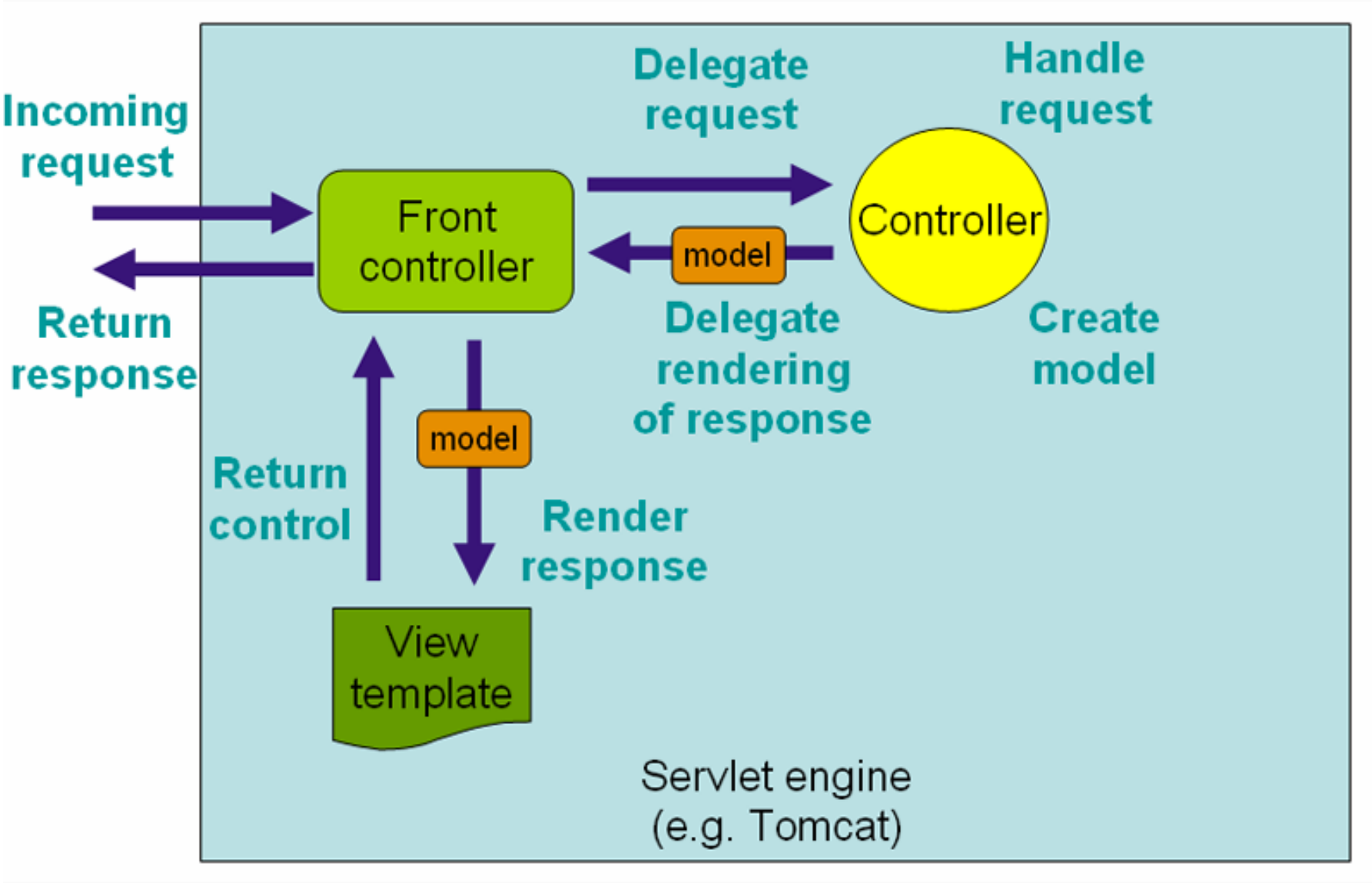
Spring WebMVC

GiớithiệuSpring Web

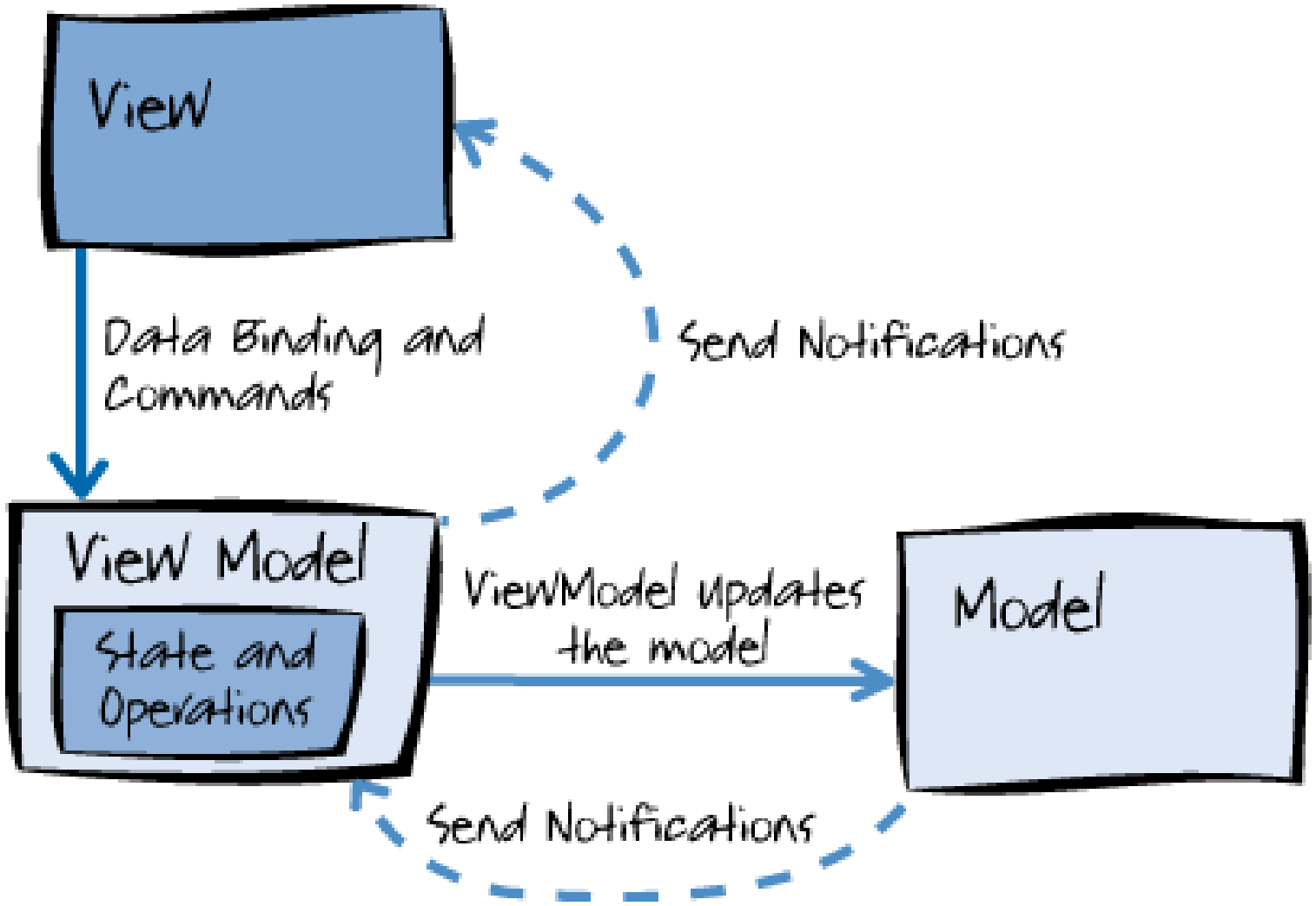


- Spring Web MVC Framework là một nền tảng mã nguồn mở phổ biến để phát triển ứng dụng Java EE
- Được cài đặt đầy đủ các đặc tính của MVC Pattern
- Cung cấp một **Front Controller** để xử lý hoặc lắng nghe mỗi khi có request tới ứng dụng

Spring MVC xử lý request



ModelView-ViewModel



DispatcherServlet



- Mỗi request đến sẽ được đón nhận và xử lý bởi FrontController
- Kế thừa từ HTTPServlet
- DispacherServlet gửi các request tới các Controller và quyết định hồi đáp bằng cách gửi lại view
- Cấu hình của DispatcherServlet trong file web.xml:

```
<servlet>
   <servlet-name>books</servlet-name>
   <servlet- class>org.springframework.web.servlet.DispatcherServlet</servle t-
class>
</servlet>
<servlet-mapping>
   <servlet-name>books</servlet-name>
   <url-pattern>*.htm</url-pattern>
</servlet-mapping>
```

Controller



- Springcontrollerxử lý các requestđể thực hiện các logic nghiệp vụ
- Annotation @RequestMapping ánh xạ một URL đến một phương thức của controller
- Ví dụ:

```
@Controller
public class CustomerController {
  @RequestMapping(value = "/input- customer")
  public String inputCustomer() {
    // do something here
    return "CustomerForm";
  }
}
```

ViewResolver



- ViewResolver là cơ chế để xử lý tầng view của Spring MVC
- ViewResolver ánh xạ tên của view sang đối tượng view tương ứng
- Có nhiều implementation khác nhau của ViewResolver:
  - AbstractCachingViewResolver
  - XmlViewResolver
  - ResourceBundleViewResolver
  - UrlBasedViewResolver
  - InternalResourceViewResolver
  - FreeMarkerViewResolver
  - ContentNegotiatingViewResolver

ModelAndView



- ModelAndView đại diện cho một view cùng với các dữ liệu sử dụng trong view đó
- ModelAndView có thể kèm theo status của Response
- Ví dụ:

```
@GetMapping("/goToViewPage")
public ModelAndView passParametersWithModelAndView()
  { ModelAndView modelAndView = new
  ModelAndView("viewPage");
  modelAndView.addObject("message", "Baeldung");
  return modelAndView;
}
```

Inversion of Control Pattern



- IoC là một nguyên lý trong phát triển phần mềm, trong đó việc điều khiển các đối tượng hoặc các thành phần của hệ thống được thực hiện bởi framework hoặc các container
- IoC cho phép framework nắm giữ quyền điều khiển luồng thực thi của hệ thống và gọi các mã nguồn khác
- Lợi ích:
  - Tách rời việc thực thi (execution) và việc triển khai (implementation)
  - Dễ chuyển đổi giữa các implementation
  - Dễ phân tách module hơn
  - Dễ kiểm thử hơn, bằng cách tách rời các thành phần riêng lẻ
- Có thể triển khai IoC thông qua một số cơ chế: Strategy design pattern, Service Locator pattern, Factory pattern, and Dependency Injection (DI)

Dependency Injection



- Dependencyinjection là một cơ chế để triển khai IoC
- Cácdependencyđượccung cấp và điềukhiển bởicontainerhoặc framework
- Thaotác“tiêm”cácđối tượng vàotrong đối tượng khácđượcthực hiện bởi container hoặc framework
- Ví dụ:

```
public class Store {
  private Item item;

  public Store() {
    item = new
    ItemImpl1();
  }
}
```

```
public class Store {
  private Item
  item; public
  Store(Item item) {
    this.item = item;
  }
}
```

Dependency Injection: Ví dụ(1/3)



```
public class A {
   public void importantMethod() {
      B b = ... // get an instance of B
      b.usefulMethod();
      ...
   }
   ...
}
```

Dependency Injection: Ví dụ(2/3)



```
public class A {
  private B b;
  public void importantMethod(){
    // no need to worry about creating B anymore
    // B b = ... // get an instance of B
    b.usefulMethod();
    ...
  }

  public void setB(B b) {
    this.b = b;
  }
}
```

Dependency Injection: Ví dụ(3/3)



```
public class A {
   private B b;

public A(B b) {
   this.b = b;
   }

   public void importantMethod(){
     // no need to worry about creating B anymore
     // B b = ... // get an instance of B
     b.usefulMethod();
     ...
   }
}
```

Spring IoC Container



- IoC container được đại diện bởi interface ApplicationContext
- Spring container chịu trách nhiệm khởi tạo, cấu hình và tổ chức các đối tượng – còn được gọi là các beans
- Spring MVC cung cấp một số triển khai:
  - ClassPathXmlApplicationContext
  - FileSystemXmlApplicationContext
  - WebApplicationContext
- Ví dụ:

```
ApplicationContext context =new ClassPathXmlApplicationContext("applicationContext.xml");
```



- Containersẽgọimộtconstructorvớicácthamsố,mỗithamsốđại diện cho một dependency
- Springnhận biếtcácthamsốthông qua kiểudữliệu,têncủathuộc tính và chỉ số của tham số
- Vídụ:

```
@Configuration
public class AppConfig { @Bean
   public Item item1() {
      return new ItemImpl1();
   }
   @Bean
   public Store store() {
      return new Store(item1());
   }
}
```

Cấu hình bean bằng XML



- Ví dụ:

```
<bean id="store" class="org.baeldung.store.Store">
  <constructor-arg type="ItemImpl1" index="0" name="item"
  ref="item1" />
</bean>
```

Setter-Based Dependency Injection



- Container gọi các phương thức setter để khởi tạo bean
- Ví dụ:

```
@Bean
public Store store() {
  Store store = new
  Store();
  store.setItem(item1
  ()); return store;
}
```

- Hoặc:

```
<bean id="store" class="org.baeldung.store.Store">
  <property name="item" ref="item1" />
</bean>
```

Field-Based Dependency Injection



- Sử dụng annotation @Autowired để tiêm các dependency

- Ví dụ:

```
public class Store {
   @Autowired
   private Item item;
}
```



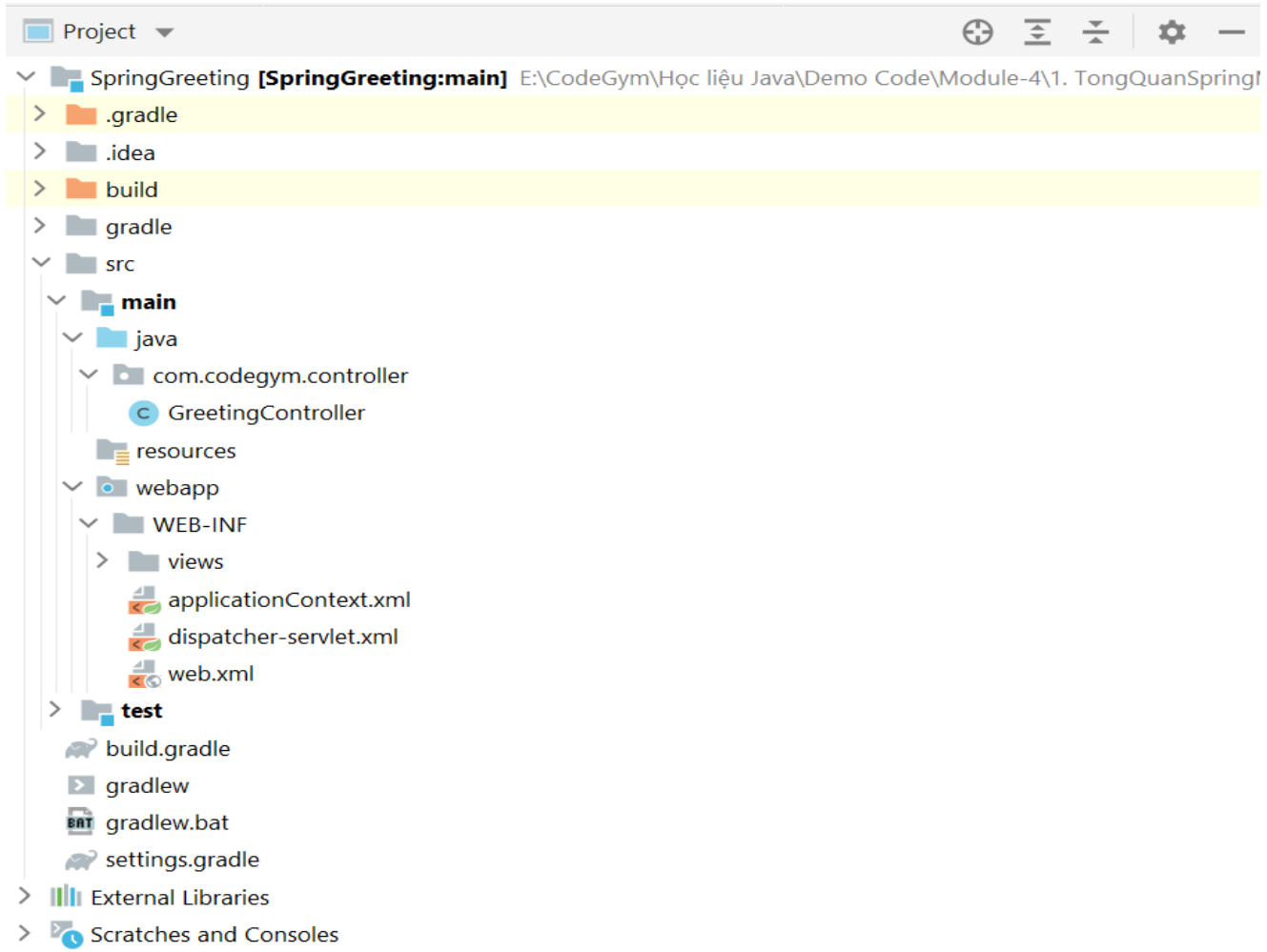
Tạo ứng dụng Spring MVC

Các cách tạo dự án Spring MVC



- Sử dụng IDE truyền thống
- Sử dụng Maven
- Sử dụng Gradle
- Sử dụng Spring Web
- Sử dụng Spring Starter Project
- Sử dụng Spring Boot

Cấu trúc thư mục



File dispatcher-servlet.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema
    /beans" xmlns:xsi="http://www.w3.org/2001/XMLSchema- instance"
    xmlns:context="http://www.springframework.org/ schema/context"
    xmlns:mvc="http://www.springframework.org/sch ema/mvc"
    xsi:schemaLocation="http://www.springframework
    .org/schema/beans
    http://www.springframework.org/schema/beans/s pring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context
    /spring-context.xsd http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/mvc/spr ing-mvc.xsd">
  <context:component-scan base-package="controllers"/>
  <mvc:annotation-driven/>
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/views/"/>
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>
```

File web.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<web-app
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XML Schema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" version="3.1">
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

Controller



```
package com.codegym.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

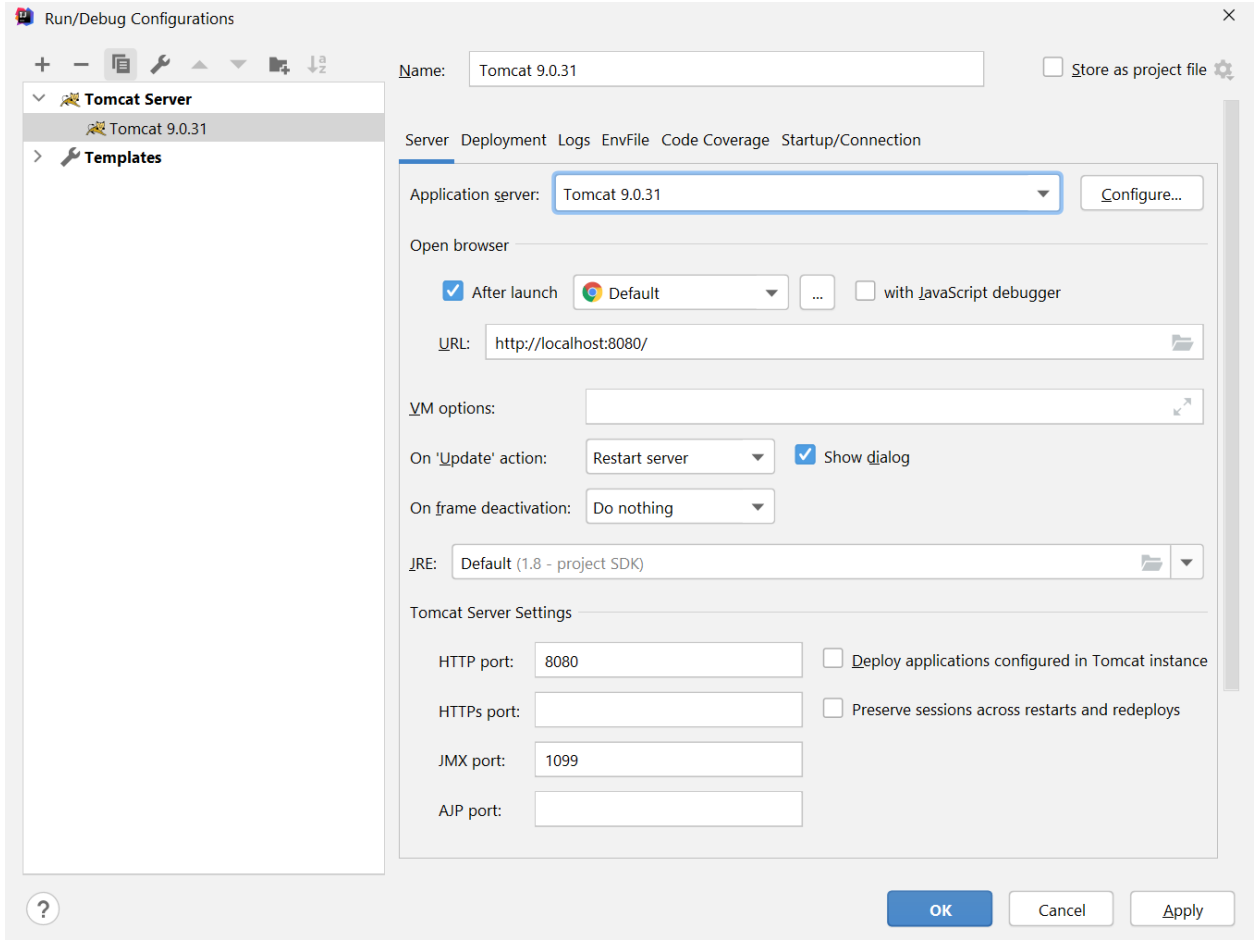
@Controller
public class GreetingController {
    @GetMapping("/greeting")
    public String greeting(@RequestParam String name, Model model) {
        model.addAttribute("name", name);
        return "index";
    }
}
```

View



```
<%@ page contentType="text/html;charset=UTF- 8" language="java" %>
<html>
<head>
 <title>Greeting</title>
</head>
<body>
 <h1>Hello: ${name}</h1>
</body>
</html>
```

Tomcat Server



Tổng kết



- Các framework giúp cho việc lập trình các ứng dụng trở nên dễ dàng hơn
- Dependency Injection là cơ chế các container hoặc framework tạo các đối tượng và cung cấp cho những nơi cần chúng
- @Autowired là annotation được sử dụng để tiêm các đối tượng
- DispatcherServlet là front controller của Spring MVC, nhận các request và chuyển đến controller tương ứng